

1. Project Summary

Module 6 refactors the existing GradCafe analytics application into a four-service microservice stack.

The Flask web tier remains responsive by publishing long-running and data-modifying work to RabbitMQ, while a separate Python worker performs the requested task and updates PostgreSQL inside a controlled transaction.

Outcome: The finished stack starts reproducibly with Docker Compose, exposes the application at localhost:8080, exposes RabbitMQ management at localhost:15672, persists PostgreSQL data in a named volume, and publishes both custom service images to Docker Hub.

2. Architecture and Service Roles

Service	Primary Role	Key Controls
web	Serves the Flask analysis page and publishes background jobs.	Binds to 0.0.0.0:8080; publishes durable, persistent messages; returns promptly.
worker	Consumes tasks and performs scraping or analytics work.	prefetch_count=1; manual ack; commit-before-ack; rollback and nack on failure.
db	Stores applicants and ingestion watermarks.	PostgreSQL health check; named volume; parameterized/composed SQL.
rabbitmq	Buffers tasks between the web and worker services.	Durable direct exchange, durable queue, binding, management UI, health check.

3. End-to-End Message Flow

Browser → Flask Web → RabbitMQ → Python Worker → PostgreSQL

A user clicks Pull Data or Update Analysis in the Flask interface.

1. The Flask endpoint calls `publish_task()` and sends a compact JSON task to the durable tasks exchange using routing key tasks.
2. RabbitMQ stores the persistent message in the durable tasks_q queue.
3. The worker consumes one message at a time because `basic_qos(prefetch_count=1)` is configured.
4. The worker opens a PostgreSQL transaction, routes the task by kind, executes the handler, and commits only on success.
5. After a successful commit, the worker acknowledges the message. On failure, it rolls back and sends `basic_nack(requeue=false)`.

4. Publisher, Consumer, and Reliability Controls

4.1 RabbitMQ Publisher

- Reads `RABBITMQ_URL` from the environment and opens a pika `BlockingConnection`.
- Declares the durable direct exchange tasks, durable queue tasks_q, and routing-key binding idempotently.
- Creates compact JSON containing kind, UTC timestamp, and payload.
- Publishes with `delivery_mode=2` so messages are persistent.
- Closes the connection in a finally block and propagates errors so Flask can report broker failures.

4.2 Worker Consumer

- Runs as a long-lived process and reconnects during short broker startup delays.
- Uses manual acknowledgements and `prefetch_count=1` to provide backpressure.
- Routes `scrape_new_data` and `recompute_analytics` through a task map.
- Processes each message inside its own database transaction.
- Acknowledges only after a successful database commit.

- Rolls back and rejects failed or malformed tasks without requeueing, which avoids infinite retry loops.

5. Database Initialization and Idempotence

At startup, the worker ensures that the required schema exists and loads the bundled applicant dataset from the read-only `/data/applicant_data.json` mount. The PostgreSQL data directory is stored in the named `postgres_data` volume, so ordinary container recreation does not erase the database.

- Applicant inserts use parameterized SQL and `ON CONFLICT DO NOTHING` to prevent duplicate records.
- The `ingestion_watermarks` table stores the last processed source position.
- The scrape handler reads the current watermark before requesting new records.
- The watermark is advanced only after successful inserts within the same transaction.
- If any part of the task fails, the transaction is rolled back and the previous watermark remains intact.

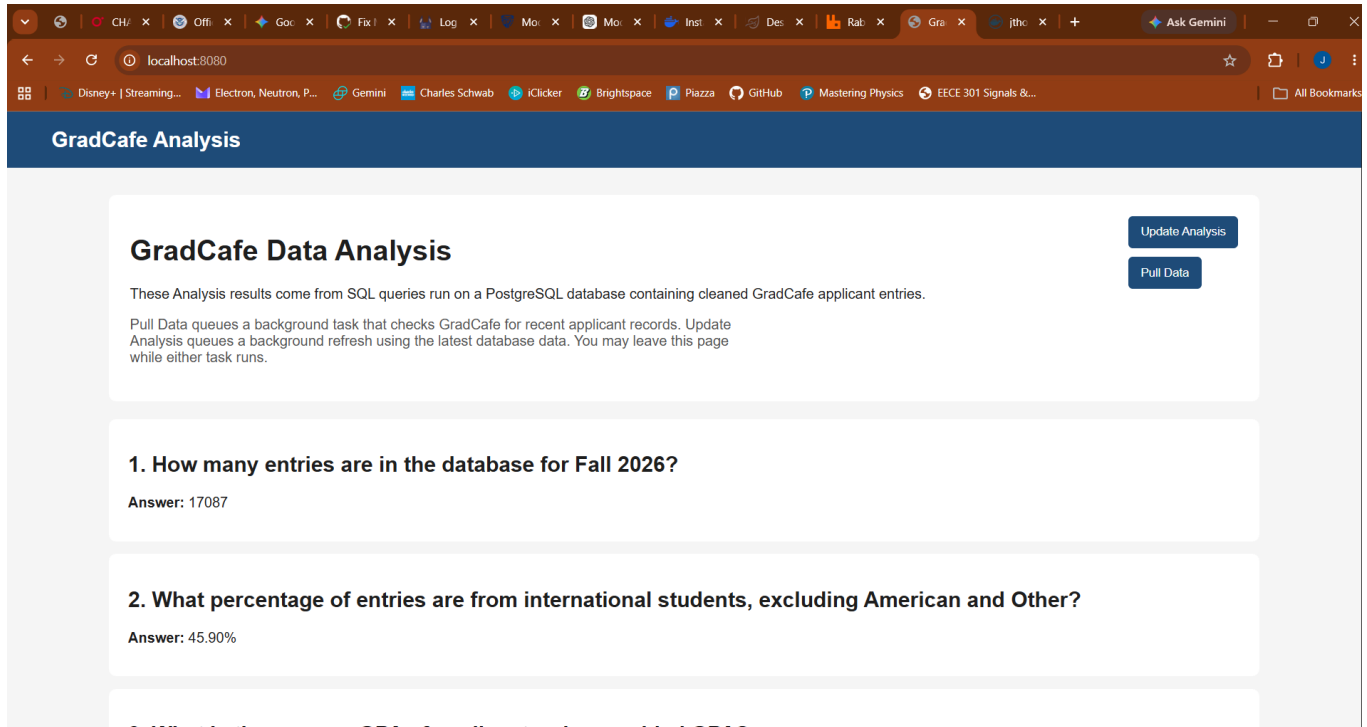
6. Docker Compose Configuration

- Defines all four required services: `web`, `worker`, `db`, and `rabbitmq`.
- Uses service health checks and `depends_on` conditions so application services wait for PostgreSQL and RabbitMQ.
- Publishes port 8080 for Flask and ports 5672/15672 for RabbitMQ AMQP and management access.
- Uses an explicit bridge network for service-to-service communication.
- Mounts applicant data read-only into the worker container.
- Runs both custom containers as a non-root application user.
- Builds from Python 3.11 slim images with pinned, service-specific dependencies.

7. Operational Verification

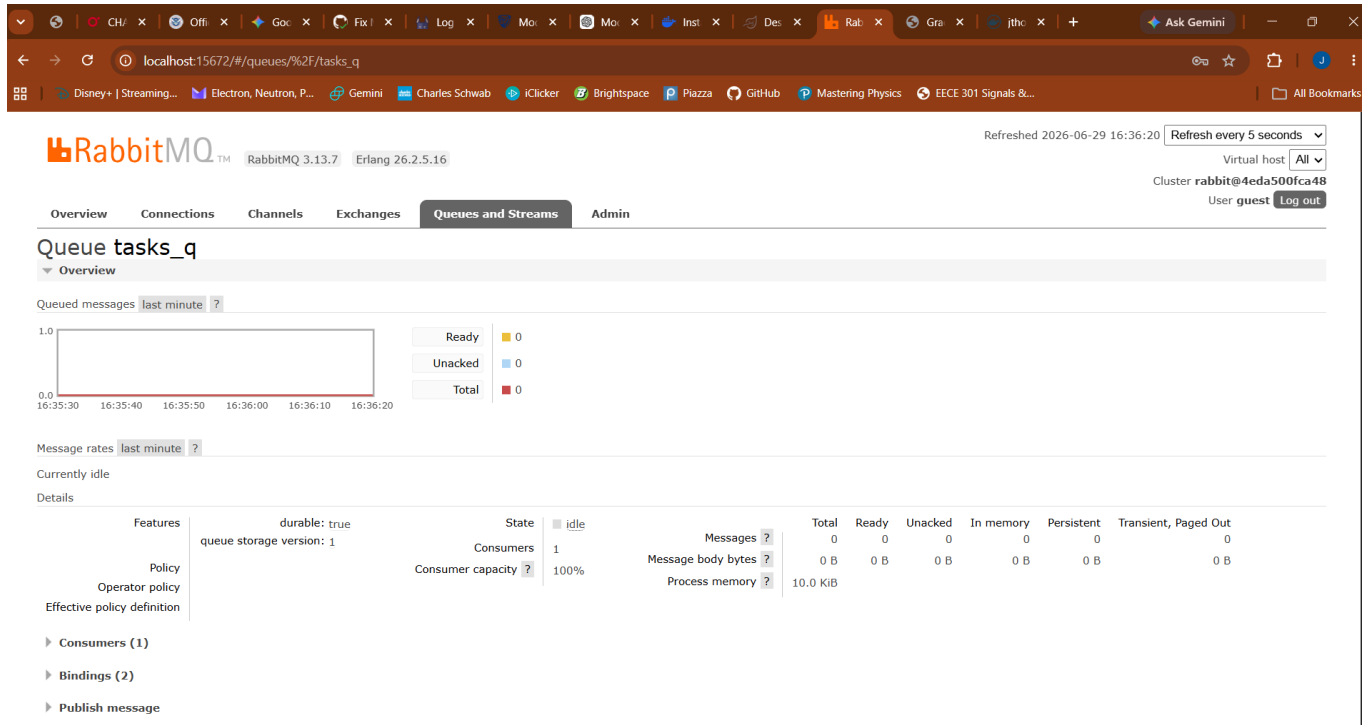
7.1 Running Flask Website

The following screenshot shows the running GradCafe analysis application at <http://localhost:8080>. The page displays PostgreSQL-backed analysis results and exposes both required task buttons. The explanatory text confirms that each button queues background work rather than blocking the browser request.



7.2 RabbitMQ Queue Verification

The RabbitMQ management view below shows the durable tasks_q queue at <http://localhost:15672>. The queue has one active consumer, and the Ready and Unacked counts are both zero. This is the expected healthy idle state after the worker has processed the queued tasks.



8. Testing, Coverage, Linting, and CI

The final source code was verified locally and through the Module 6 GitHub Actions workflow. The workflow contains separate jobs for Pylint, Pytest with coverage enforcement, and Docker Compose validation/build.

Verification	Result
Pytest	50 tests passed
Coverage	100.00%
Pylint	10.00/10
GitHub Actions	All Module 6 jobs passed
Docker Compose	Configuration validated and web/worker images built successfully

CI workflow: Module 6 Microservices CI runs only for Module 6 paths and does not replace the preserved workflows for prior modules.

9. Published Images and Reproducibility

Both custom application images are published in a public Docker Hub repository: https://hub.docker.com/r/jthom358/module_6

- jthom358/module_6:web
- jthom358/module_6:worker

A clean machine with Docker Desktop and Docker Compose can run the stack from the module_6 directory with:

```
docker compose up -d --build
```

Published images can also be pulled directly:

```
docker pull jthom358/module_6:web
```

```
docker pull jthom358/module_6:worker
```

10. Submission Links

Private GitHub repository: https://github.com/jthom358/jhu_software_concepts

Public Docker Hub repository: https://hub.docker.com/r/jthom358/module_6